

メディアコンピューティング2026

第9回 パターン検出

目次

- テンプレートマッチングによるパターン検出
- エッジ情報とヒストグラムによるパターン検出
 - チャンファーマッチング
 - カラーヒストグラムによるマッチング

テンプレートマッチング

■パターンマッチング

- 画像中におけるパターンの存在や位置を検出すること
 - ◆画像の視覚的特徴や画素値そのもの

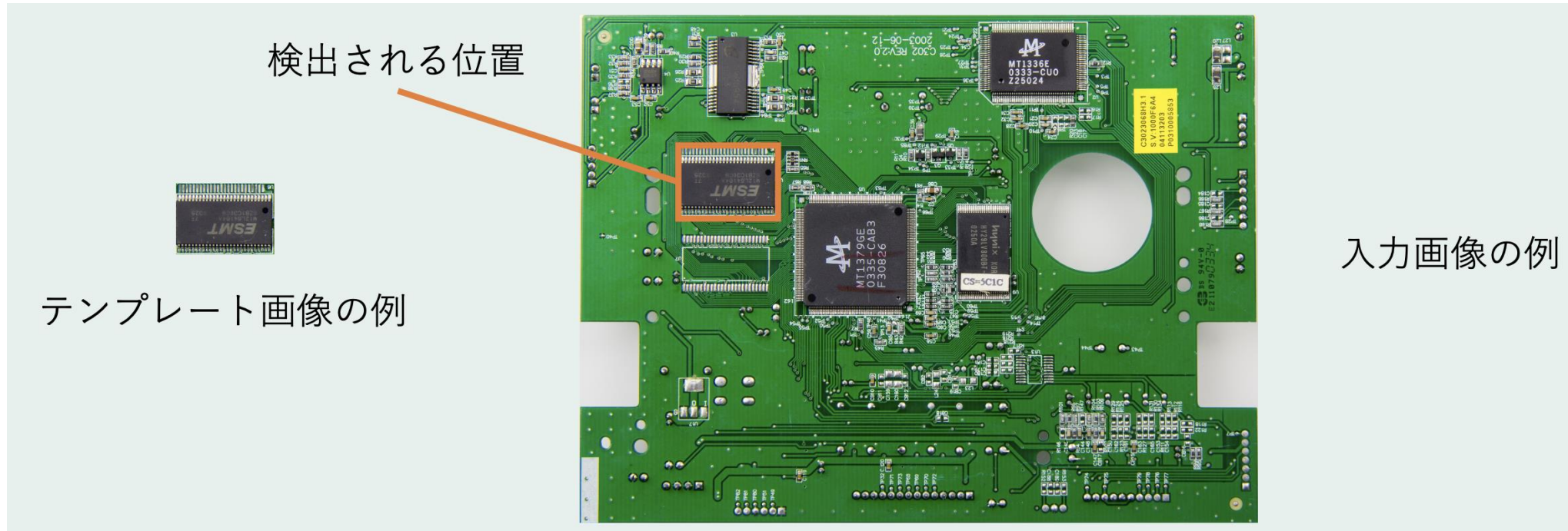
■テンプレートマッチング

- あらかじめ用意した標準パターンを使用
 - ◆テンプレート（画像）と呼ばれる
- 製造現場における外観検査などで利用

テンプレートマッチングの活用例

■プリント基板の組み立て検査

- 入力画像：基板全体を撮影した画像
- テンプレート：指定されたICを撮影した画像



テンプレートマッチングの仕組み

- 1. テンプレートを入力画像の上で走査
 - 操作方法の代表的なものはラスタスキャン
 - テンプレートを走査させる範囲を探索範囲という
- 2. 入力画像上の各位置における類似度（または相違度）を算出
- 3. 最大の類似度（最小の相違度）の位置を検出

類似度または相違度の計算方法

■相違度

- SSD (Sum of Squared Difference)
- SAD (Sum of Absolute Difference)

■類似度

- NCC (Normalized Cross-Correlation)
- ZNCC (Zero mean NCC)

SSD

■画素値の差の二乗和

- 値が小さいほど類似
- 完全に一致したとき 0

$$R_{SSD} = \sum_{j=0}^{N-1} \sum_{i=0}^{M-1} (I(i,j) - T(i,j))^2$$

- M : テンプレートの高さ
- N : テンプレートの幅
- $I(i,j)$: 入力画像の位置 (i,j) における画素値
- $T(i,j)$: テンプレートの位置 (i,j) における画素値

SAD

■画素値の差の絶対和

- 値が小さいほど類似
- 完全に一致したとき 0

$$R_{SAD} = \sum_{j=0}^{N-1} \sum_{i=0}^{M-1} |I(i,j) - T(i,j)|$$

- M : テンプレートの高さ
- N : テンプレートの幅
- $I(i,j)$: 入力画像の位置 (i,j) における画素値
- $T(i,j)$: テンプレートの位置 (i,j) における画素値

NCC

■画像をベクトルとみなしたときの内積

➤値が大きいほど類似（範囲は-1から1）

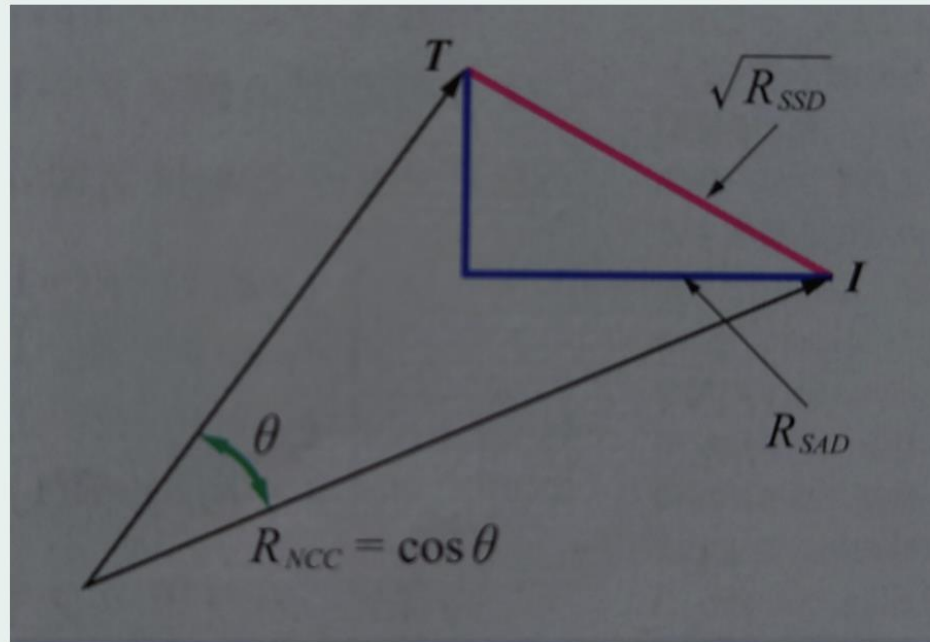
$$R_{NCC} = \frac{\sum_{j=0}^{N-1} \sum_{i=0}^{M-1} I(i,j)T(i,j)}{\sqrt{\sum_{j=0}^{N-1} \sum_{i=0}^{M-1} I(i,j)^2 \sum_{j=0}^{N-1} \sum_{i=0}^{M-1} T(i,j)^2}}$$

- M : テンプレートの高さ
- N : テンプレートの幅
- $I(i,j)$: 入力画像の位置 (i,j) における画素値
- $T(i,j)$: テンプレートの位置 (i,j) における画素値

SSD, SAD, NCCの関係

テンプレート $T(i,j)$, 対象画像 $I(i,j)$ をベクトル I, T と考えると

- SAD: ベクトル I, T の先端間のマンハッタン距離
- SSD: ベクトル I, T の先端間のユークリッド距離
- NCC: ベクトル I, T のなす角 θ の余弦 $\cos\theta$



ZNCC

■平均値を 0 にしたNCC

- NCCを正規化したもの
- 値が大きいほど類似（範囲は-1から1）

$$R_{ZNCC} = \frac{\sum_{j=0}^{N-1} \sum_{i=0}^{M-1} (I(i,j) - \bar{I})(T(i,j) - \bar{T})}{\sqrt{\sum_{j=0}^{N-1} \sum_{i=0}^{M-1} (I(i,j) - \bar{I})^2 \sum_{j=0}^{N-1} \sum_{i=0}^{M-1} (T(i,j) - \bar{T})^2}}$$

- $\bar{I}, \bar{T}$: テンプレート，入力画像の領域内の画素の平均値
- M : テンプレートの高さ
- N : テンプレートの幅
- $I(i,j)$: 入力画像の位置 (i,j) における画素値
- $T(i,j)$: テンプレートの位置 (i,j) における画素値

ZNCCの特徴

- ゲイン変動に強い

- 明るさやコントラストの違い

- 計算量が多い

探索の高速化

■探索範囲内の全ての位置で類似度を計算

➤多くの計算時間が必要



■探索を高速化する手法が提案

➤残差逐次検定法

➤粗密探索法

残差逐次検定法（SSDA法）

- SADを計算するときに、しきい値を超えたら探索を打ち切る

$$R_{SAD} = \sum_{j=0}^{N-1} \sum_{i=0}^{M-1} |I(i, j) - T(i, j)|$$

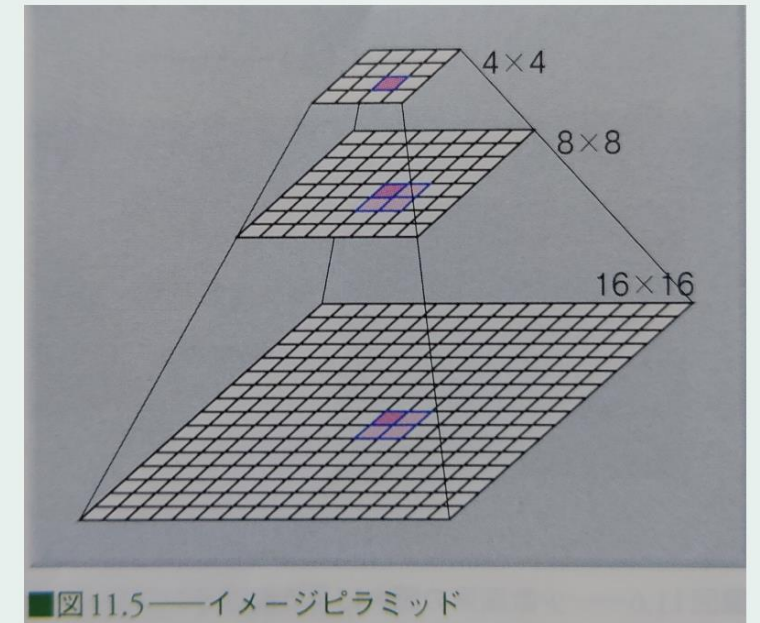
- しきい値は動的に決定
 - それまで計算したSADの最小値とする
- 常に真の最小値に到達することが保証される
- 大幅に計算時間を短縮できる

粗密探索法

画像情報を何段階かの解像度で表現，解像度の疎密性を利用

手順

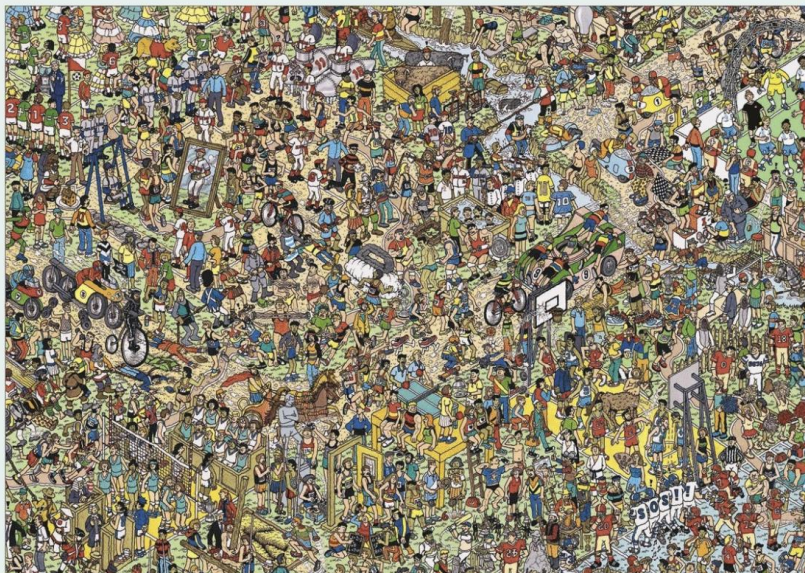
1. 入力画像からイメージピラミッドを構成
入力画像に対して平均化により
画像解像度をつぎつぎに下げる
2. テンプレートの低解像度画像も用意
3. 低解像度画像から順に探索する



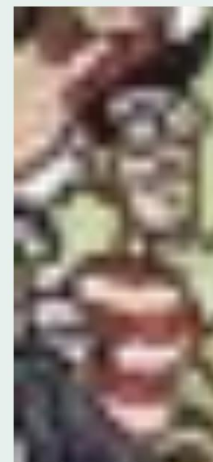
演習 1

■パターンマッチングでウォーリーを探せ！

- mc09-1.cppを編集
- find_wally.jpgからwallyを検出



find_wally.jpg



wally.png

演習 1 で使用する関数 (1 / 2)

テンプレートマッチングにより類似度(相違度)マップを得る関数
`cv::matchTemplate(sourcelmage, templatemage, resultImage, <METHOD>)`

- `sourcelmage`: 入力画像
- `templatemage`: テンプレート
- `resultImage`: 類似度(または相違度)マップ
- `METHOD`: 類似度(または相違度)の計算方法
 - `cv::TM_SQDIFF_NORMED`: SSD
 - `cv::TM_CCORR_NORMED`: NCC
 - `cv::TM_CCOEFF_NORMED`: ZNCC

演習 1 で使用する関数 (2 / 2)

最大値, 最小値およびその位置を返す関数

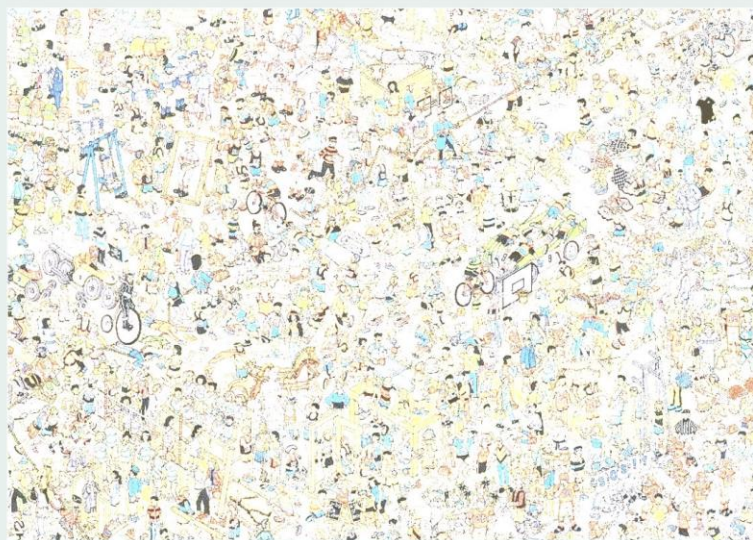
```
cv::minMaxLoc(resultImage, &minVal, &maxVal,  
               &minLoc, &maxLoc)
```

- resultImage: 類似度(または相違度)マップ
- minVal, maxVal: 類似度(または相違度)の最小値, 最大値
- minLoc, maxLoc: 類似度(または相違度)が最小値, 最大値をとる箇所の座標

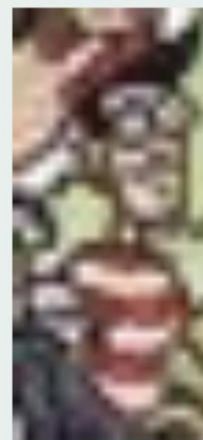
演習 2

■ ZNCCが明るさの変化に強いことを確かめよう

- 画像をbright_wally.jpgに代えて，各指標で検出
- ZNCCの効果を確認



bright_wally.jpg



wally.png

目次

- テンプレートマッチングによるパターン検出
- エッジ情報とヒストグラムによるパターン検出
 - チャンファーマッチング
 - カラーヒストグラムによるマッチング

テンプレートマッチングとの違い

- テンプレートマッチングでは画素値そのものをテンプレートとして使用

- 画像の**特徴量**を使用

 - エッジ情報

 - カラーヒストグラム

チャンファーマッチング

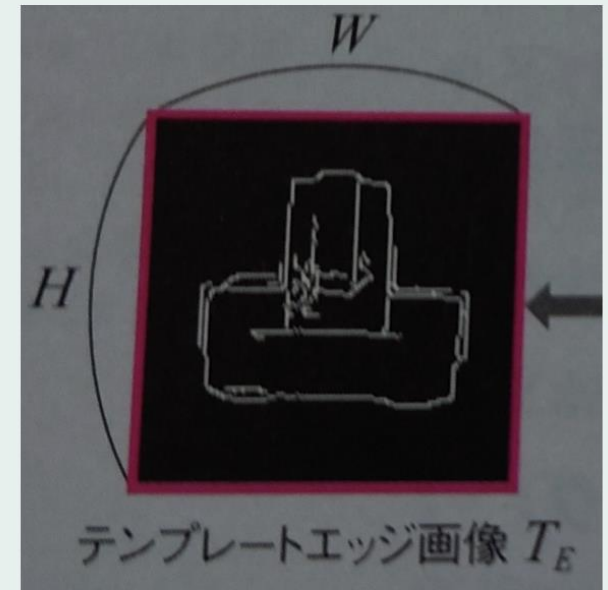
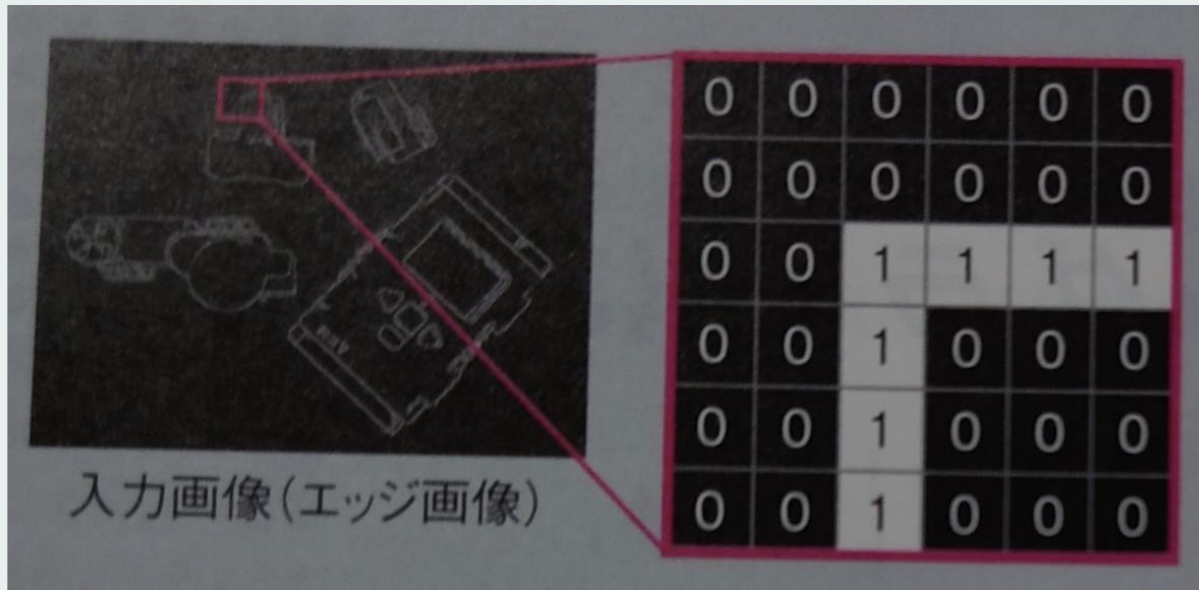
■入力画像とテンプレート間のエッジ相違度に基づいて探索

手順

1. 入力画像のエッジ画像を作成
2. 1.から距離変換画像を作成
3. テンプレートのエッジ画像(テンプレートエッジ画像)を作成
4. 2.と3.の相違度を計算
5. 4.の相違度に対する勾配を求める

エッジ画像の作成

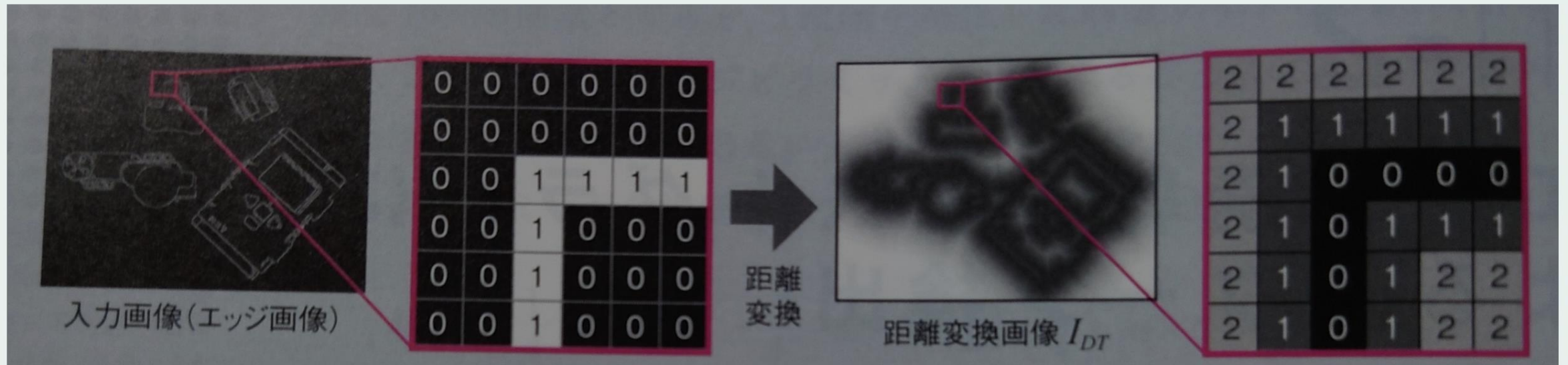
ケニーのエッジ検出アルゴリズムなどのエッジ検出法により、入力画像およびテンプレートからそれぞれエッジ画像を作成



距離変換画像の作成

入力画像のエッジ画像に距離変換処理をおこない、
距離変換画像 I_{DT} を作成

もっとも近いエッジからの距離を書き込む



相違度の計算

入力画像上の画素 (x, y) について, 相違度 $S(x, y)$ を計算する

$$S(x, y) = \sum_{j=0}^H \sum_{i=0}^W T_E(i, j) I_{DT}(x + i, y + j)$$

- H : テンプレートの高さ
- W : テンプレートの幅
- $I_{DT}(i, j)$: 距離変換画像 I_{DT} の位置 (i, j) における画素値
- $T_E(i, j)$: テンプレートエッジ画像 T_E の位置 (i, j) における画素値

勾配の計算

入力画像上の画素 (x, y) について勾配 $(\frac{\partial}{\partial x} S(x, y), \frac{\partial}{\partial y} S(x, y))$ を計算する

$$\frac{\partial}{\partial x} S(x, y) = \sum_{j=0}^H \sum_{i=0}^W T_E(i, j) \frac{\partial}{\partial x} I_{DT}(x + i, y + j)$$

$$\frac{\partial}{\partial y} S(x, y) = \sum_{j=0}^H \sum_{i=0}^W T_E(i, j) \frac{\partial}{\partial y} I_{DT}(x + i, y + j)$$

- H : テンプレートの高さ
- W : テンプレートの幅
- $I_{DT}(i, j)$: 距離変換画像 I_{DT} の位置 (i, j) における画素値
- $T_E(i, j)$: テンプレートエッジ画像 T_E の位置 (i, j) における画素値

テンプレートの移動先決定と移動

1. テンプレートの移動先 (x', y') を決定する
$$(x', y') = (x + \frac{\partial}{\partial x} S(x, y), y + \frac{\partial}{\partial y} S(x, y))$$
2. テンプレートを (x', y') へ移動させ、
相違度の計算から繰り返す

距離変換画像はエッジまでの距離を表している

 テンプレートはエッジ方向に移動する

勾配が 0 になったら

■相違度がしきい値以下

- そのときの位置を検出位置として決定
 - ◆ (=マッチングの完了)

■相違度がしきい値以上

- 初期点を変更し，相違度の計算から繰り返し

チャンファーマッチングの利点

■相違度の勾配を利用

➤全探索をする必要がない



➤高速にマッチングできる

カラーヒストグラムとマッチング

- カラーヒストグラム: 色番号に対する頻度を表したもの
- RGB色空間における色番号 $i = R \times Q \times Q + G \times Q + B$
 - R, G, B : 各成分の値
 - Q : 各成分の最大色数

マッチングへの利用

- カラーヒストグラムが十分類似
- 同一パターンとして評価できる

類似度の算出方法

- ヒストグラムインタセクション
- バタチャリヤ係数

ヒストグラムインタセクション

1. 各色番号における頻度の大小を比較
2. 小さい方の頻度の総和を類似度 $\rho(p, q)$ とする

$$\rho(p, q) = \sum_{i=1}^n \min(p_i, q_i)$$

- n : 色番号の個数
- p, q : ヒストグラム
- p_i, q_i : ヒストグラムにおける色番号 i の頻度
- ヒストグラムの分布が類似しているほど $\rho(p, q)$ は高くなる
- $\rho(p, q)$ の範囲は-1から1

バタチャリヤ係数

各色番号における頻度を掛け合わせ、
その総和を類似度 $\rho(p, q)$ とする

$$\rho(p, q) = \sum_{i=1}^n \sqrt{p_i \times q_i}$$

- n : 色番号の個数
- p, q : ヒストグラム
- p_i, q_i : ヒストグラムにおける色番号 i の頻度
- ヒストグラムの分布が類似しているほど $\rho(p, q)$ は高くなる
- $\rho(p, q)$ の範囲は-1から1

アクティブ探索

■ カラーヒストグラムの類似度を全探索

- 計算コストが高い



■ 周辺領域の類似度計算を省き，探索を効率化

- 類似度が大きく低い箇所周辺の類似度は十分に低い
- そのような箇所を丸々スキップして高速化
 - ◆ ただし，最適解を保障．探索範囲のみを削減

類似度計算の省略

- 入力画像中の任意の領域 A, B について考える
 - A に対する $A \cap B$ の重なり率と B のテンプレートとの類似度から A とテンプレートとの類似度の上限を求められる
 - A とテンプレートとの類似度の上限が小さいならば,
 A とテンプレートは類似していない
-  A とテンプレートの類似度計算を省略

類似度の上限

領域 A とテンプレート M との類似度 $S(A, M)$ の上限

$$S(A, M) \leq \frac{\min(S(B, M)|B|, |A \cap B|) + |A - B|}{|A|}$$

- A, B : 入力画像中の任意の領域
- M : テンプレート
- $A \cap B$: A, B の共通領域
- $A - B$: A の領域から B の領域を除いた領域
- $|A|$: A の画素数

類似度と重なり率

- $S(B, M)$ が低く, $A \cap B$ の重なり率が高い
 - $S(A, M)$ の上限値は小さい
 - A とテンプレートの類似度を計算しない
- $S(B, M)$ が高い
 - $S(A, M)$ の上限値は大きい
 - A とテンプレートの類似度を計算する

表 11.1 — 類似度 $S(B, M)$ と重なり率 $A \cap B$ の変化による $S(A, M)$ の上限値の変化

類似度 $S(B, M)$	共通領域 $A \cap B$ の重なり率		
	90%	50%	10%
0.8	0.9	1.0	1.0
0.5	0.6	1.0	1.0
0.1	0.2	0.6	1.0

演習 3

■チャンファーマッチングでパターン検出しよう

➤mc09-2.cppを編集



aquarium.jpg



fish.jpg

演習 3 の流れ

- 入力画像を二値化
- 二値化した画像からエッジ画像を作成(`cv::Sobel`)
- エッジ画像を白黒反転
- 白黒反転したエッジ画像から距離変換画像を作成
- テンプレート画像を二値化
- 二値化した画像からエッジ画像を作成
- 距離変換画像とエッジ画像でマッチング(`cv::TM_CCORR`)

演習 3 で使用する関数 (1 / 3)

■ 白黒反転する関数

```
cv::bitwise_not(sobelImage,sobelImage)
```

sobelImage: ソーベルフィルタで得られたエッジ画像

演習 3 で使用する関数 (2 / 3)

■ 距離変換関数

```
cv::distanceTransform(sobelImage, distImage,  
                     <DISTANCETYPE>, 5)
```

- sobelImage: ソーベルフィルタで得られたエッジ画像
- distImage: 距離変換画像
- DISTANCETYPE: 距離の計算方法
(演習では cv::DIST_L2)

演習 3 で使用する関数 (3 / 3)

■正規化する関数

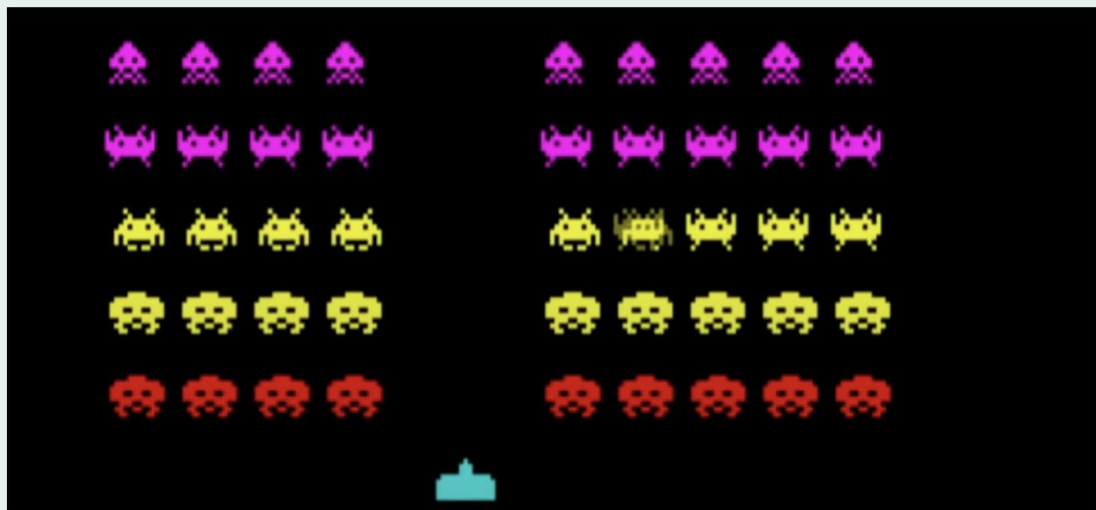
```
cv::normalize(distImage, normDistImage, 0.0, 255.0,  
              <NORMTYPE>, 0)
```

- distImage: 距離変換画像
- normDistImage: 正規化したdistImage
- 0.0, 255.0: 0から255までにするという処理
- NORMTYPE: 正規化の方法
(演習ではcv::NORM_MINMAX)

課題

■動画でパターンマッチングしよう

➤処理結果の動画を提出



game.mp4



player.png